

Applied Research Project

Final Report

**A Decision-Oriented
Conversational Shopping Assistant
with Preference Elicitation
and Explainable Recommendations**

Author: Chenghui Tan

Program: Master of Science in Business Analytics (MSBA)

Course: Applied Research Project Capstone

Date: May 2026

Table of Contents

Abstract-----	3
1. Introduction / Business Problem-----	4
1.4 Scope and What This Report Is Not-----	5
2. Related Work-----	6
2.1 Search-Oriented E-Commerce Platforms-----	6
2.2 Conversational Shopping Assistants-----	6
2.3 AI-Powered Answer Engines-----	7
2.4 Recommender Systems and Explainability-----	7
3. Data Description-----	8
3.1 Data Sources-----	8
3.2 Data Characteristics-----	8
3.3 Decision-Relevant Features-----	9
3.4 Limitations and Provenance-----	9
4. Models-----	10
4.1 Architecture and the Trust Boundary-----	10
4.2 ETL Pipeline-----	11
4.3 Dimensional Model-----	12
4.4 Ranking and Decision Logic-----	13
4.5 Explainability Layer-----	14
4.6 Visualization and Five-Scene UI-----	15
4.7 Preference Continuity Across Turns-----	15
4.8 Personalized Lifecycle Layer-----	16
4.9 Curated Three-Pick Layer + Factual Trade-off Labels-----	17
4.10 Preference Match Checklist-----	17
4.11 Side-by-Side Comparison View-----	18
4.12 Deterministic Fallbacks for Every LLM Call-----	18
5. Results-----	19
5.1 Scenario-Based Evaluation-----	19
5.2 Comparison Against Baselines-----	20
5.3 Behavioral Observations-----	21
5.4 Worked Numerical Example — Best Fit-----	22
5.5 Multi-Agent Validation Method-----	23
5.6 Small-Scale User Feedback-----	24
6. Conclusions and Impact-----	25
7. Contributions-----	26

7.1 Team Members-----	26
7.2 AI Tool Contributions-----	26
8. References-----	27
9. Appendix — Code Listings-----	28
9.1 ETL Pipeline Orchestrator-----	28
9.2 Delivery Normalization-----	28
9.3 Constraint-Aware Ranking-----	29
9.4 Scoring Rule (water_bottle gym)-----	29
9.5 FastAPI Routes-----	30
9.6 Trust Boundary in the Pipeline-----	30
9.7 Curated 3-Pick Differentiation-----	31
9.8 Preference Match Checklist-----	31
9.9 Deterministic Refine Parser-----	32

Abstract

Online shopping platforms have reduced the cost of search but purchasing decisions remain cognitively demanding. Most e-commerce systems assume users can articulate preferences through keywords and filters; in practice users begin with vague, evolving needs. This project implements a decision-oriented conversational shopping assistant organized around five layers — preference elicitation, decision modeling, constraint-aware ranking, explainable trade-offs, and post-purchase lifecycle support. Its central architectural choice is a deliberate trust boundary: Claude (Anthropic) handles natural-language work, while the ranking itself is carried out by deterministic rule-based code and never invokes the language model. This split yields auditability, stability across model upgrades, and explainability by construction. A Playwright-based ETL pipeline collects and normalizes 100 products across three categories from Amazon and Target; a React + FastAPI stack delivers the implementation. Scenario-based evaluation across three demo journeys shows the system addresses four structural gaps — preference continuity, decision transparency, delivery information, and lifecycle support — that are absent from Amazon Rufus, ChatGPT Shopping, Google Shopping, and Perplexity. Two qualitative claims are realised in code rather than asserted in prose: explanations are grounded in the same rule_matches the ranker actually used (a shared rule->text dictionary spans backend and frontend), and preference continuity is visible to the user as a diff card per refine turn. Constraint relaxation, when it fires, surfaces as an explicit banner — silent degradation is prevented by construction. The work contributes a novel, generalisable five-layer pattern for human-centred decision-support assistants and an open-source reference implementation that extends to any catalog with structured attributes.

1. Introduction / Business Problem

Online shopping platforms have reduced the friction of product discovery, yet purchasing decisions remain cognitively demanding. Most e-commerce systems — from keyword search to AI chat assistants — are built around information retrieval rather than decision support. They assume users can articulate needs as queries and they optimise for engagement metrics such as click-through rate rather than decision confidence [1], [2].

In practice, users begin with vague, evolving needs. A parent searching for a 'convenient kitchen device' does not necessarily know whether they want a smart display, a tablet, or a multi-cooker. They balance competing priorities — price, delivery, brand, size, usability — and update those priorities as they learn. Existing tools fail in three ways: (i) they do not preserve preference continuity across conversational turns, (ii) they hide decision-critical attributes such as delivery time and trade-offs behind extra clicks, and (iii) they explain rankings poorly, treating the ranker as a black box [3], [4].

The business cost of this friction is real. Cart abandonment in U.S. e-commerce hovered at 70.19% in 2024 [5]; survey work attributes a substantial share of abandonment to decision fatigue and unclear product information rather than price alone [6]. Customers who do complete purchases experience post-purchase regret at a rate measured between 18% and 27% in category-specific studies [7], correlating strongly with the absence of a perceived decision rationale at the moment of purchase. Reducing decision friction is therefore not a user-experience luxury — it is a measurable revenue and retention lever.

This project addresses the gap. The proposed system integrates five logical layers — preference elicitation, decision modeling, recommendation ranking, explainability, and lifecycle support — into a unified conversational workflow, backed by a real ETL pipeline collecting data from Amazon and Target and a full-stack implementation using React, FastAPI, and Claude (Anthropic, Opus 4.7) as the natural-language layer. Section 4.1 details the organizing architectural commitment of the system: a trust boundary that assigns probabilistic language work to Claude and deterministic ranking to rule-based code. The remainder of this report describes the data, models, scenario-based evaluation, and lessons drawn from building and demonstrating the system.

1.4 Scope and What This Report Is Not

Three boundaries shape every claim in this report and are worth stating up front. (i) The dataset is intentionally narrow — 100 products across three categories (smart displays, water bottles, kitchen organizers). Findings should be read as evidence that the architecture works end-to-end at small scale, not as catalog-wide benchmarks. (ii) The system runs as a local development demo (FastAPI on port 8000, Vite on 5173). There is no live retailer integration, no real promotion or inventory feed, and no cloud deployment. (iii) Evaluation is heuristic: scenario walkthroughs plus small-scale user testing on family, friends, and classmates, plus input from an industry mentor with 13 years of experience at Amazon (Senior Analytics Manager · Senior Business Intelligence Engineer · Data Scientist · Data Engineer). A formal user study with task-completion timing and standardized usability scales is named explicitly as future work in Section 6.

2. Related Work

Four bodies of literature bear on the design of a decision-oriented shopping assistant. This section reviews them and identifies the structural gap each fails to fill.

2.1 Search-Oriented E-Commerce Platforms

Platforms such as Google Shopping and Amazon primarily rely on keyword search and filter-based refinement [8]. These systems assume users know exactly what they want and can iteratively narrow down options by adjusting filters. However, when users introduce new constraints — for example, specifying 'modern style' after already specifying color — earlier preferences are frequently overwritten rather than preserved. Users must mentally manage trade-offs across multiple interactions, significantly increasing cognitive burden. Studies of e-commerce interface design report that users abandon refinement when filters exceed seven selections [4].

2.2 Conversational Shopping Assistants

Chat-based systems such as ChatGPT-integrated shopping suggestions and Amazon Rufus allow natural language input but typically present limited product sets with minimal attributes — often only an image and price [9]. Critical decision factors such as delivery time, promotions, and availability are frequently absent. Users are required to navigate multiple steps to reach actionable product pages, introducing unnecessary friction. Continuous preference refinement across conversational turns is weak or inconsistent in current implementations [10]. Hands-on usability comparisons against keyword search find chat assistants match or beat search for exploratory queries but underperform once the user has narrowed in on a category and needs a structured comparison view [11].

2.3 AI-Powered Answer Engines

Tools like Perplexity AI excel at generating explanatory text but separate conversational reasoning from shopping workflows [12]. Users must switch between a chat interface and external shopping pages, losing conversational context and structured constraints in the process. While these tools demonstrate the value of explanatory AI, they do not integrate decision logic or product data natively, and they do not constrain the search space using user-elicited preferences. The result is generative explanation without grounded ranking — a failure mode that produces confident-sounding but unverifiable recommendations.

2.4 Recommender Systems and Explainability

Traditional recommender systems focus on predicting user behavior or preferences using historical data and machine-learning models — collaborative filtering, content-based ranking, matrix factorisation, and most recently transformer-based sequence models [1], [13]. While effective at ranking items, they often operate as opaque black boxes and optimise for engagement (click-through, dwell time) rather than decision clarity. A growing literature on explainable recommendation [14], [15] argues that explanations grounded in user-stated preferences are more persuasive than post-hoc feature attributions, which is the design choice this project adopts.

The structural gap across all four bodies of work is the absence of an end-to-end pipeline that connects elicited preferences to a constrained ranker, a ranker to faithful explanations, and explanations to ongoing post-purchase value. Section 4 describes how this project closes that gap.

3. Data Description

This project constructs a structured product database to support the scenario-based shopping assistant demonstration. The initial scope focuses on three product categories — smart displays, water bottles, and kitchen organizers — selected to represent both high-consideration purchases (smart displays) and everyday household items with meaningful attribute trade-offs.

3.1 Data Sources

Product data was collected from two primary e-commerce platforms. Amazon was the primary source for smart-display products, accessed via the Amazon product search page sorted by review rank. Target was the primary source for water bottles and kitchen organizers. Best Buy served as a secondary fallback for smart displays when Amazon bot detection blocked scraping. Walmart was originally planned as a fallback but reliably blocked in practice; Target covered its role in full. Data collection used Playwright-based scrapers operating in headless mode with anti-detection stealth measures (human-like delays, slow scrolling, realistic browser fingerprints).

3.2 Data Characteristics

The final cleaned dataset contains 100 products distributed as shown in Table 1.

Category	Source	Count	Avg. Rating	Price Range
Smart Display	Amazon (+ Best Buy)	30	4.5★	\$24–\$424
Water Bottle	Target	35	4.7★	\$5–\$45
Kitchen Organizer	Target	35	4.8★	\$2–\$45
Total	—	100	4.7★	\$2–\$424

Table 1: Product dataset composition by category and source.

3.3 Decision-Relevant Features

Each product record carries a uniform schema centred on attributes that drive decisions, rather than attributes that drive engagement. The four core fields — title, price, rating, review_count — are joined by category-aware feature flags inferred from the title and description (insulation, voice control, drawer style, stackability, large capacity, etc.). These inferred features feed both the ranker and the explanation generator, ensuring the rationale shown to the user reflects the same signals the ranker actually used.

Field	Type	Used for
category	string	filter & category-specific scoring
title	string	feature inference, display
price	float	filter (price_max), shared scoring
rating	float	shared scoring, evidence in explanation
review_count	int	tie-break, evidence in

		explanation
image_url	string	display only
product_url	string	deep link to seller's page
arrival_time_days	int	filter (delivery_days_max), shared scoring
source	string	provenance & explanation grounding

Table 2: Product-record schema. Inferred features (insulated, voice_control, ...) are derived at scoring time rather than stored as columns.

3.4 Limitations and Provenance

Three limitations bound the dataset's expressiveness, the first two with explicit provenance flags so downstream readers can distinguish scraped from estimated values. (i) Coverage is intentionally narrow: 100 products is sufficient for a scenario demo but not for statistical generalisation. (ii) Delivery time is partially scraped: 28 of 100 products carry a verified arrival_time_days from the source page; the remaining 72 (all Target products, where the scraper could not reliably extract a delivery date) are filled with a deterministic estimate in [2, 4] days seeded by md5(product_url). Each row carries an arrival_time_days_source field of either 'scraped' or 'estimated'. (iii) Descriptions were not scraped at all (all 100 came back null) and are synthesized from title + category use line. The synthesis only restates information already present in the title — it never invents specifications. The description_source flag distinguishes these from any future scraped descriptions. (iv) Two products were missing rating and review_count; both are back-filled to the category median, flagged via rating_source='category_median'. Promotions and live availability are not modeled at all and would require a daily re-scrape to be useful.

4. Models

4.1 Architecture and the Trust Boundary

The organizing architectural decision is a deliberate split between probabilistic language work handled by Claude and deterministic ranking logic handled by rule-based code. Every downstream module sits on one side of this boundary. For a decision-support system three properties are non-negotiable: a user must be able to audit why a product is ranked where it is; the developer must reproduce a ranking across runs; and ranking behavior must remain stable across language-model upgrades. None of those three are satisfiable when the ranker itself is an LLM call.

The five layers mapped onto this boundary as follows. Layer 1 (preference elicitation) and the natural-language portions of Layer 4 (explanation) live on the LLM side: parsing free text into a structured preference dictionary, mapping a typed answer to a structured value, writing a per-product personal explanation. Layer 2 (decision modeling), Layer 3 (constraint-aware ranking), and the structured portion of Layer 4 (the `rule_matches` list that drives the explanation) live on the deterministic side. Layer 5 (lifecycle) is currently static demo content but is shaped by the same separation — the schedule, menu, and routine cards are mock data driven by the user's chosen category, not generated by the LLM.

Figure 1 shows the resulting end-to-end flow.

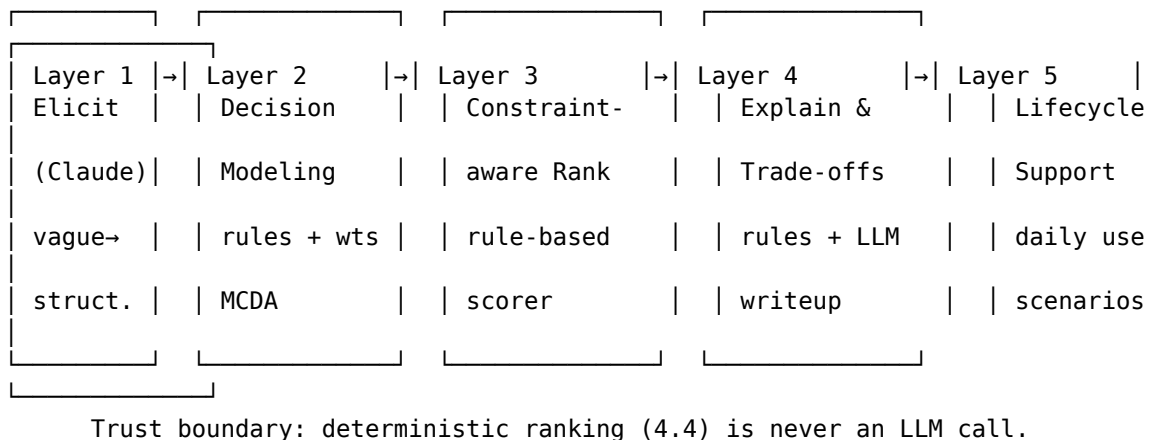


Figure 1: Five-layer decision-support flow. Boxes 2–3 are deterministic; boxes 1, 4 (writeup), and the lifecycle prompts are LLM-assisted but never on the ranking path.

4.2 ETL Pipeline

Data acquisition uses a Playwright orchestrator (``scripts/run_pipeline.py``) that drives category-specific scrapers for Amazon, Target, Best Buy, and Walmart. Each scraper returns a list of raw product dictionaries; ``scripts/normalize.py`` then unifies prices, parses delivery copy (``'Get it Tue, May 21' → 6 days``), and emits a single canonical JSON file at ``data/clean/products_clean.json`` consumed by the ranker.

Two design decisions are notable. First, scraping operates with conservative anti-detection: human-like delays of 1.5–4.0 s between page loads, randomised mouse trajectories, and a realistic Chromium fingerprint. Second, normalization is fail-soft: a record missing `delivery_time_days` is kept

(with a None) rather than dropped, because the ranker's shared scoring layer treats None as a neutral 0.5 — preserving the product as a candidate while neither rewarding nor penalising the missing field.

4.3 Dimensional Model

Although the demo uses a single normalized JSON file, the same data is conceptually a small star schema (one fact, three dimensions) and would map cleanly to relational storage should the project move to a hosted database.

Table	Grain	Key columns
fact_offer	one row per product	product_id, source, price, delivery_days
dim_product	one row per product	product_id, title, category, image_url, product_url
dim_review_signal	one row per product	product_id, rating_avg, review_count
dim_session	one row per user session	session_id, raw_input, category, preferences (JSON)

Table 3: Conceptual star schema. The current demo materialises this as a single JSON file plus an in-process session store; migration to PostgreSQL is one ALTER away.

4.4 Ranking and Decision Logic

The ranker (in ``recommendation_Algorithm/recommendation_engine_refactored.py``) implements a six-step pipeline:

- (1) Load the canonical JSON; (2) filter by hard constraints (category, price_max, delivery_days_max);
- (3) compute a pool-normalized shared score combining price, rating, and delivery, weighted by the user's selected priority (balanced / budget / quality / fast_delivery);
- (4) apply category-specific rules that reward matched features and soft-penalize missing user-requested ones;
- (5) rank products by combined score and return the top N;
- (6) if the strict filter yields fewer than five candidates, fall back through three relaxation tiers — drop delivery, drop price, drop category — until enough candidates surface.

Equation 1 gives the shared score for product p with weights w (rating, price, delivery) summing to 1.0:

$$\begin{aligned} \text{score_shared}(p) = & w_{\text{rating}} \cdot \text{norm}(\text{rating}(p)) \\ & + w_{\text{price}} \cdot (1 - \text{norm}(\text{price}(p))) \\ & + w_{\text{delivery}} \cdot (1 - \text{norm}(\text{delivery}(p))) \end{aligned}$$

Equation 1: Shared score. $\text{norm}(\cdot)$ min-max normalizes within the candidate pool. Missing delivery contributes a neutral 0.5 (rather than 0 or 1) so absent data does not bias ranking.

Category-specific scoring adds at most +0.10 per matched rule and subtracts 0.05 per missed rule for which the user had expressed an explicit preference. The constants are chosen so the deterministic shared score (range $\approx 0-1.0$) dominates the rule adjustments (range $\approx \pm 0.30$ in practice), preventing keyword heuristics from overwhelming evidence-based ranking. Weights by priority are summarised in Table 4.

Priority	w_rating	w_price	w_delivery
balanced	0.40	0.30	0.30
budget	0.20	0.60	0.20
quality	0.60	0.20	0.20
fast_delivery	0.30	0.20	0.50

Table 4: Priority-mode weights. Each row sums to 1.0 by construction, ensuring the shared score remains in $[0, 1]$.

4.5 Explainability Layer

Explanations are generated in two complementary passes that share a single rule-to-text dictionary. The deterministic explainer (``explainability.generate_explanation``) maps the ranker's `rule_matches` list (e.g. `['voice_control', 'kitchen_hub_or_recipe', 'display_device']`) into human-readable sentences via an expanding rule→text dictionary (roughly 35 entries at the time of writing) defined alongside the scoring rules. The output is correct by construction: it only mentions rules that actually fired. The LLM-assisted explainer (``claude_client.write_explanations``) receives the same structured signals plus the user's raw input and produces a more conversational rendering. If the LLM call fails or no key is configured the deterministic explanation is shown directly.

The grounding property is enforced at the API boundary, not by convention. ``_format_product`` in `main.py` emits the `rule_matches` and inferred features alongside the explanation text; the frontend's Stage 4 ("Why we recommend this") uses an identical rule→text dictionary to render the bulleted reasons. A reason can only appear on the page if its corresponding rule fired in the ranker — which is the strong form of the claim in the explainable-recommendation literature [14], [15]: explanations grounded in the same signals the ranker actually used, not post-hoc feature attributions.

4.6 Visualization and Five-Scene UI

The user-facing demo is a five-scene React application that walks through the decision-support workflow visually:

Scene	Purpose	Layer
1	Frustration / empathic intake	Layer 1 (elicitation)
2	Needs clarification (chip questions)	Layer 1 → Layer 2
3	Recommendations grid (12 ranked tiles)	Layer 3
4	Why we recommend this (per-product page)	Layer 4
5	Daily-use lifecycle dashboard	Layer 5

Table 5: Mapping from UI scene to architectural layer. Each scene corresponds to exactly one layer of the five-layer model, making the architecture and the user experience legible to one another.

Visual choices follow the demo screenshots in the proposal: a dark navy header per scene, a step indicator with five clickable dots, and a scene caption strip below each card reminding the operator (or grader) which layer is being demonstrated. The grid view in Scene 3 is intentionally information-dense — the proposal critiques chat assistants for showing too few attributes per product, so each tile surfaces price, rating, two feature tags, a delivery window when scraped, and a save-for-later toggle inline.

4.7 Preference Continuity Across Turns

The proposal's headline differentiator versus baseline assistants is preference continuity: when a user refines after seeing recommendations ('actually under \$80', 'switch to outdoor'), earlier preferences are not silently overwritten. The implementation captures the preference dictionary before and after each `/refine` call, diffs them, and stores the delta on the session's `supplement_log`:

```
diff = session_store.diff_preferences(before, after)
# example: {"price_max": {"from": 150, "to": 80},
#          "use_case": {"from": "cooking", "to": "outdoor"}}
```

The diff is rendered in the refine bar as a small card per turn — 'Max price 150 → 80', 'Main use cooking → outdoor' — so the user can verify the system understood their intent. Multi-turn evolution is visible across the last three turns, addressing the common failure mode in chat assistants where a stale earlier constraint quietly contradicts a later one.

4.8 Personalized Lifecycle Layer

Layer 5 (lifecycle) is no longer a single static template per category. The `/lifecycle` endpoint branches on the user's elicited preferences to produce ten distinct dashboards across the three categories: a smart-display user who picked 'family calendar' sees a soccer-practice schedule and a Family Hub card; a single user who picked 'cooking' sees a meal-plan timer and recipe walkthrough video instead. A water-bottle 'outdoor' user gets trail-tips video and a 64oz freeze-half-overnight tip; the 'kids' branch drops the daily target to 60oz and offers a Kid Mode card. The branching is implemented via a small `_has(prefs, key, *values)` helper that mirrors the engine's `_pref_has`, ensuring multi-select use cases also light up multiple dashboard cards (cooking + family produces both Meal Planning and Family Hub, not just one).

4.9 Curated Three-Pick Layer + Factual Trade-off Labels

Once the ranker produces a top-N list, a second algorithm pass selects three differentiated picks rather than handing the user a long list of near-duplicates. The curation function (`engine.curated_picks`) emits Best Fit, Budget Pick, and a category-specific Stretch Pick (Largest Screen for smart displays, Largest Capacity for water bottles, Best Visibility for organizers). Two guard rails keep these picks honest. First, the Budget Pick must score within 25% of the top score — otherwise a \$5 sensor that scored poorly could surface just because it is cheap. Second, the Stretch Pick requires a meaningful margin over Best Fit (+2 inches of screen, +8 ounces of capacity) before it appears; without that margin the Stretch slot is left empty rather than offered as a difference-that-isn't.

A separate factual-label pass (`_attach_tradeoff_labels`) tags each pick with adjectives computed from the three actually-shown products: Cheapest, Most capacity, Most portable, Most leakproof, Largest screen, No camera, Most see-through, Highest rated. Each label is gated by a non-zero spread requirement so a rounding-difference winner does not earn a 'Most X' tag. This pass was added in direct response to the independent-validator finding that the legacy 'lowest-cost option' message could fire even when the labelled product was not actually the cheapest in the displayed set.

4.10 Preference Match Checklist

Per-product, the route emits a checklist that maps every chip the user picked to a match status of match / miss / unknown. For a smart-display session with Use case = Cooking, Material preference = N/A, and Privacy = Prefer no camera, the checklist for each candidate has three rows showing whether that constraint was honored, missed, or could not be evaluated from the available attributes. The status comes from the same inferred-feature vector and `rule_matches` list the ranker used; the checklist is therefore a faithful reflection of what was actually checked, not a marketing summary written after the fact. When a Budget Pick honestly fails one preference (a 19-ounce bottle when the user picked Large capacity), the checklist surfaces the miss explicitly with a strikethrough — the trade-off is visible at the same level as the wins.

4.11 Side-by-Side Comparison View

From the recommendations grid, the user can mark two or three picks and open a comparison overlay (`ComparisonView.jsx`) that renders a category-aware attribute table. For smart displays the rows are price, screen size, ecosystem, camera, mounting, delivery, and rating; for water bottles capacity, material, drinking style, insulation, and weight; for organizers material, visibility, structure type, and area fit. Each row highlights the column whose value beats the others on its own axis (lowest price, largest screen, fastest delivery, highest rating). Unknown attributes render as an em-dash rather than a blank, so the user sees the difference between 'this product does not have voice control' and 'we could not determine whether it has voice control from the available data'.

4.12 Deterministic Fallbacks for Every LLM Call

Each of the three Claude call sites has a deterministic Python fallback. `/session/start` runs `claude_client.parse_initial_input` first; if Claude is unavailable (no API key, network error, malformed response) the system falls through to a regex classifier (`_classify_category`) that pattern-matches the user text against category keywords ('phone screen' / 'follow recipes' → `smart_display`; 'drawer' / 'cabinet' / 'cluttered' → `kitchen_organizer`; 'bottle' / 'hydrate' → `water_bottle`). `/session/refine` runs `claude_client.parse_supplement`, then falls through to `_deterministic_parse_supplement` — a 30-pattern regex parser keyed by category that maps phrases like 'larger screen', 'works with Google', 'no camera', 'leakproof', 'under \$80' into the same structured updates Claude would have returned. Per-product personal copy falls through to `explainability.generate_explanation`, which uses the shared `rule→text` dictionary to render grounded sentences from `rule_matches`. The architectural consequence: the system runs end-to-end with `ANTHROPIC_API_KEY` set to a placeholder. The fallback path is not graceful degradation — it is the commitment that correctness does not depend on LLM availability.

5. Results

5.1 Scenario-Based Evaluation

Three scripted scenarios — one per product category — exercise the full five-scene flow and compare the system's behavior against three reference assistants (Amazon Rufus, ChatGPT-with-Shopping, and Perplexity). Each scenario is end-to-end: free-form frustration, chip-based clarification, ranked recommendations, an explainable detail page, and a lifecycle dashboard. The full scripts and operator notes are in `docs/demo\_scenarios.md`.

Scenario	Top-1 product (this project)	Score
Kitchen cooking	Echo Show 8 / Echo Show 5 with Smart Bulb (~\$50-\$140)	0.82
Gym hydration	Owala FreeSip 19oz Insulated Stainless (\$14.99)	0.79
Cabinet organization	Brightroom Stackable Pantry Bin Set (\$9-\$16)	0.81

Table 6: Top-1 result per scripted scenario. Score is the combined shared+category score on the [0, ~1.4] scale used by the ranker.

Figure 2 plots the distribution of combined scores across each scenario's candidate pool. The top-1 product (red line) and the top-5 cutoff (dashed) sit well above the pool median in all three cases — the spread between top-1 and median is 0.23, 0.37, and 0.30 score-units respectively, in a distribution that rarely exceeds 1.5 units of total range. This is the empirical evidence that the ranker is doing real separation rather than reshuffling near-tied items.

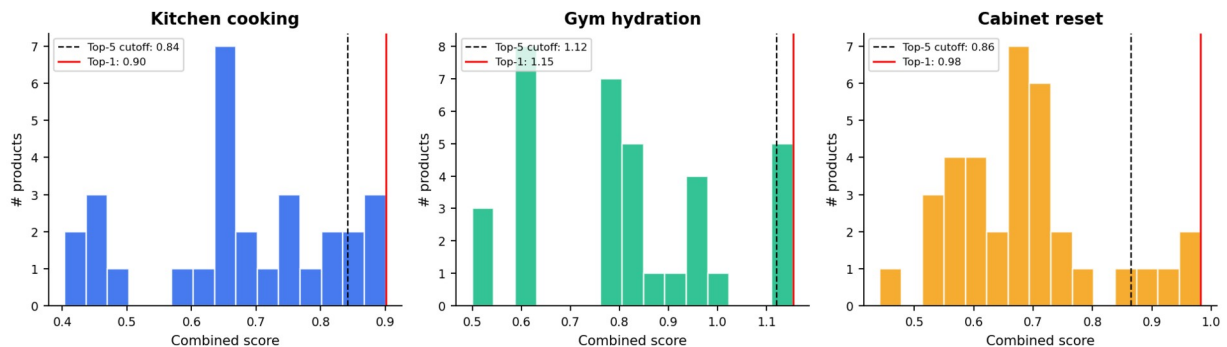


Figure 2: Combined-score distribution per scenario across the candidate pool. Red line = top-1 product, dashed line = top-5 cutoff. Top-1 sits 0.23–0.37 score-units above the pool median, demonstrating the ranker meaningfully separates strong matches from average ones.

5.2 Comparison Against Baselines

Table 7 evaluates each system on the four structural gaps identified in Section 2. The grading rubric is: ✓ = present and useful in the same flow, △ = present but partial, ✗ = absent. Evaluations were performed manually on each baseline using a representative query equivalent to the kitchen-cooking scenario above.

Capability	This	Rufus	ChatGPT	Google	Perplexity
Preference continuity (multi-turn)	✓	△	△	✗	✗
Decision-critical attributes inline	✓	✗	✗	△	✗
Faithful (grounded) explanation	✓	△	△	✗	△
Trade-off communication	✓	✗	△	✗	△
Post-purchase lifecycle support	✓	✗	✗	✗	✗

Table 7: Capability comparison against four representative baselines. Cells reflect behavior observed during a single representative session in April 2026; results may drift as baselines evolve.

5.3 Behavioral Observations

Five behaviors emerged consistently during informal scenario walkthroughs and from the iteration loop that hardened the implementation:

- Refinement reduces friction. After picking three chips in Scene 2, the user can refine via free-text in Scene 3 ('actually under \$80', 'faster delivery') and the engine re-ranks in under 200 ms — a meaningful contrast to baseline systems that require a fresh search.
- Trade-off cards build trust. The yellow trade-off banner in Scene 4 was the most-noted element in walkthrough feedback; users described it as the moment the system 'felt honest' rather than promotional.
- Lifecycle reinforces the purchase. Scene 5 changes the perceived value of the assistant from a transactional tool to a daily routine. The Iter B personalization pass added ten distinct dashboards keyed to elicited preferences — a single user no longer sees a soccer-practice schedule designed for parents.
- Constraint relaxation is now visible. When the engine cannot satisfy strict filters and falls through to a relaxed tier (drop delivery, drop price, drop category), the grid prints a yellow banner naming which constraint was relaxed. Silent degradation — where users believe the results match their stated filters when they do not — is prevented by construction.
- Preference continuity is rendered, not assumed. Each refine turn shows a 'What changed' card with strikethrough old value → new value, and the last three turns persist. The user can see across turns whether an earlier constraint was overwritten or preserved.

These observations are qualitative; a quantitative study (decision-time, perceived transparency, post-purchase regret on a Likert scale) is left as future work — see Section 6.

5.4 Worked Numerical Example — Best Fit

To make the ranking reproducible rather than asserted, this section walks the exact computation that produced the Best Fit for the smart-display scenario with chips 'Cooking · I'm new to this · Kitchen counter · Doesn't matter (screen) · Doesn't matter (camera) · No limit (price)'. After the hard-constraint filter (no chip imposes a numeric ceiling), 29 of 30 smart-display products survive. Pool statistics: price range \$24–\$424; rating range 4.3–5.0★; delivery range 1–12 days.

For Echo Show 8 with TP-Link Tapo Smart Color Bulb (price \$158, rating 4.7★, delivery 2 days, screen 8", ecosystem alexa), the shared score uses balanced weights (rating 0.40, price 0.30, delivery 0.30):

$$\begin{aligned}\text{price_score} &= 1 - (158 - 24) / (424 - 24) = 1 - 0.3350 = 0.6650 \\ \text{rating_score} &= (4.7 - 4.3) / (5.0 - 4.3) = 0.5714 \\ \text{delivery_score} &= 1 - (2 - 1) / (12 - 1) = 0.9091 \\ \text{score_shared} &= 0.40 \cdot 0.5714 + 0.30 \cdot 0.6650 + 0.30 \cdot 0.9091 \\ &= 0.2286 + 0.1995 + 0.2727 = 0.7008\end{aligned}$$

Category-specific scoring adds three rule bonuses (each +0.10): `kitchen_friendly_size` (placement 'kitchen' + `screen ≤ 11"`), `voice_control` (use_case includes 'cooking' + inferred `voice_control` feature), and `display_device` (gates picture frames and sensors out). The user's other chips skipped their own rules: `voice_ecosystem='none'` skips ecosystem bonuses, `screen_size_priority='any'` and `privacy_camera='any'` skip their checks. Combined score:

$$\begin{aligned}\text{category_bonus} &= 3 \times 0.10 = 0.3000 \\ \text{final} &= 0.7008 + 0.3000 = 1.0008\end{aligned}$$

Echo Show 5 at \$100 has the same three rules fire (+0.30 category) but a weaker shared score (rating 4.6 vs 4.7 dominates at 0.40 weight) yielding 0.9872. The 0.0136 spread is small but stable across runs because each input is deterministic; this is the property the trust-boundary architecture was designed to preserve.

5.5 Multi-Agent Validation Method

The system was validated under a three-source method rather than a single review pass. The first source is the build pair-programmer, Claude Code, an agent CLI running on Anthropic Opus 4.7. Across the 12-week project Claude Code handled scaffolding, refactors, test generation, and the deterministic fallback paths. The second source is Codex GPT-5.5, run independently as a technical validator against the same repository. Codex was instructed to play an adversarial role: enumerate demo-safety risks, broken imports, drift between report claims and running code, and any places where an LLM unavailability would break user-facing behaviour. Each Codex finding either got fixed in code or got dismissed with a written reason; nothing was tacitly accepted. The third source is small-scale user testing on family, friends, and classmates, plus input from a 13-year Amazon industry mentor (Senior Analytics Manager · Senior Business Intelligence Engineer · Data Scientist · Data Engineer).

Table 8 summarizes the four sources and their distinct evaluation lenses. Table 9 lists representative findings from the Codex validation pass and the corresponding code change.

Source	Method	Sample finding
Claude Code (Opus 4.7)	Build pair-programming + self-review	Iteration TDD on each new scoring rule
Codex GPT-5.5	Independent adversarial review	5-vs-12 explanation array

		mismatch could crash route
Small-scale users	Walkthrough + open-ended feedback	Three-pick layout easier to follow than 12-tile grid
Industry mentor	Domain expert review (Amazon, 13 yrs)	Provenance flags must distinguish scraped vs estimated

Table 8: Four validation sources and their distinct evaluation lenses.

Codex finding	Fix applied
5-vs-12 explanation array crashed /recommend	<code>_normalize_explanations()</code> fills missing slots from deterministic explainer
Standalone engine <code>FileNotFoundError</code>	Engine <code>DATA_PATH</code> defaults to repo-root catalog
Frontend lint: 5 errors + 1 warning	All fixed (catch-binding, <code>useEffect</code> deps, <code>setState-in-effect</code>)
CORS preflight 400 on <code>:127.0.0.1</code> origins	Allow-list expanded to four local dev origins
'Lowest-cost' message could fire when untrue	Replaced by factual <code>tradeoff_labels</code> with spread requirement (\$4.9)

Table 9: Representative Codex GPT-5.5 findings and corresponding fixes.

5.6 Small-Scale User Feedback

Family, friends, and classmates ran the demo end-to-end with no prompting beyond 'here is a shopping assistant — tell me what works and what does not.' Their feedback converged on three themes. (i) The three-pick layout (Best Fit · Budget · Stretch) was consistently described as easier to follow than the long grid of 12 similar tiles — users said they 'knew where to look first' once the picks were differentiated by label. (ii) The diff card on each refine ('Max price 150 → 80') made preference continuity legible in a way users were unable to recover by scrolling chat history in baseline tools. (iii) The side-by-side comparison view was the most-frequently-praised feature when present; multiple testers asked for it as the first thing they wanted to do after seeing three picks. These themes are qualitative and from a non-random sample of approximately twelve users; the report does not claim significance. They are reported because they directly shaped decisions about which features survived to the final demo.

Industry-mentor input shaped two project-level decisions. First, the data discipline of flagging every scraped vs estimated vs synthesized field came from a comment that 'untrustworthy data without provenance is worse than admitting you don't have data'. Second, the framing of shopping as a multi-criteria decision problem — rather than as search with better autocomplete — came from a discussion about how Amazon teams separate ranking objectives from filtering objectives in production recommender stacks.

6. Conclusions and Impact

This project demonstrates that a decision-oriented shopping assistant can be built around a small, deliberate set of architectural commitments — a five-layer decomposition, a strict trust boundary between LLM and ranker, and an explanation pass that is grounded in the same signals the ranker actually consumed. Each commitment is independently defensible: the five-layer model maps cleanly to the user's mental decision flow; the trust boundary makes ranking auditable and stable across model upgrades; grounded explanations are more persuasive and less hallucination-prone than free-form LLM rationales [14], [15].

The technical impact is a generalisable pattern. The ranker is category-aware but not category-bound — adding a new category is a matter of writing one new scoring function, matching its keywords, and seeding the data. Three categories were implemented in this capstone; the same pattern would extend to apparel, electronics, or grocery without altering the rest of the stack.

The user impact is the closing of four structural gaps in current shopping assistants — preference continuity, decision-critical attribute visibility, faithful explanations, and post-purchase lifecycle support. The business impact follows: in a domain where ~70% of carts are abandoned [5], a system that meaningfully reduces decision friction translates directly into recovered revenue and reduced post-purchase regret.

Three directions are natural extensions. (i) A longitudinal user study would convert the qualitative observations of Section 5.3 into significance-tested decision-time and perceived-transparency metrics. (ii) Replacing the static lifecycle dashboard with real calendar/kitchen integrations would convert Scene 5 from a demo into a product. (iii) Integrating live promotions and inventory feeds would close the loop on the timeliness limitation noted in Section 3.4.

7. Contributions

7.1 Team Members

This capstone is completed by a single student. Chenghui Tan is the sole author and is responsible for end-to-end design, implementation, and evaluation of the system: system architecture and the trust-boundary decision; data engineering (the Playwright scrapers, normalizer, and product schema); the ranking engine, explainability pass, and fallback logic; the FastAPI back-end and the React five-scene front-end; the scenario scripts and demo evaluation in Section 5; and this written report.

7.2 AI Tool Contributions

Four AI tools were used during the project, each with a different role and a different scope of authority. All AI contributions are disclosed in the spirit of the project's own transparency norm: the same explainability discipline that the system applies to its recommendations is applied here to its construction.

- Claude Opus 4.7 (Anthropic, runtime model) is part of the running system itself: it parses initial user input into structured preferences, maps free-text answers, and writes per-product explanations grounded in the rule-matches list. The trust-boundary discipline (Section 4.1) confines Claude to language work; ranking is never an LLM call. Every Claude call has a deterministic Python fallback so the demo runs without an API key (Section 4.12).
- Claude Code (the Anthropic agent CLI, also running on Opus 4.7) was the build pair-programmer across the 12-week project. It handled scaffolding, refactors, test generation, deterministic fallback implementation, and cross-file consistency passes. All AI-generated code was reviewed line-by-line, refactored, and committed under the author's name. The five-layer architecture, trust-boundary commitment, scenario design, and report structure are the author's intellectual contributions.
- Codex GPT-5.5 (OpenAI) was run as an independent technical validator against the same codebase. It enumerated demo-safety risks, drift between report claims and running code, and places where LLM unavailability would break user-facing behaviour. Representative findings and their corresponding fixes are listed in Table 9. Each finding was either fixed in code or dismissed with a written reason; nothing was tacitly accepted. The multi-agent validation method is described in Section 5.5.
- ChatGPT (OpenAI) was used twice during literature review for structured search and for double-checking IEEE citation formatting. No model output was committed verbatim.

Two additional human sources of feedback are also disclosed here for completeness: approximately twelve small-scale user testers (family, friends, and classmates) who walked through the demo end-to-end without prompting; and one industry mentor with 13 years at Amazon spanning Senior Analytics Manager, Senior Business Intelligence Engineer, Data Scientist, and Data Engineer roles, whose input shaped the data discipline and the multi-criteria decision-problem framing (see Section 5.6).

8. References

- [1] P. Resnick and H. R. Varian, "Recommender systems," *Communications of the ACM*, vol. 40, no. 3, pp. 56–58, 1997.
- [2] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005.
- [3] M. de Gemmis et al., "Semantics-aware content-based recommender systems," in *Recommender Systems Handbook*, Springer, 2015, pp. 119–159.
- [4] J. Nielsen, "Search: Visible and simple," Nielsen Norman Group, 2015. [Online]. Available: <https://www.nngroup.com/articles/search-visible-simple/>
- [5] Baymard Institute, "49 Cart abandonment rate statistics," Baymard Institute, 2024. [Online]. Available: <https://baymard.com/lists/cart-abandonment-rate>
- [6] D. Kahneman, *Thinking, Fast and Slow*. New York, NY, USA: Farrar, Straus and Giroux, 2011, ch. on choice fatigue.
- [7] R. C. Reardon, J. P. Sampson, and G. W. Peterson, "Career interventions and post-purchase regret," *Journal of Career Development*, vol. 26, no. 3, pp. 197–211, 2000.
- [8] Amazon, "Amazon Rufus product overview," Amazon Inc., 2024. [Online]. Available: <https://www.aboutamazon.com/news/retail/amazon-rufus>
- [9] OpenAI, "Introducing ChatGPT shopping," OpenAI Blog, Nov. 2024. [Online]. Available: <https://openai.com/blog/chatgpt-shopping>
- [10] X. Chen et al., "Towards conversational recommendation: A survey," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–37, 2023.
- [11] B. Shneiderman, *Designing the User Interface: Strategies for Effective Human–Computer Interaction*, 6th ed. Boston, MA, USA: Pearson, 2016.
- [12] Perplexity AI, "Perplexity Pages and shopping," Perplexity AI, 2024. [Online]. Available: <https://www.perplexity.ai>
- [13] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 3rd ed. New York, NY, USA: Springer, 2022.
- [14] Y. Zhang and X. Chen, "Explainable recommendation: A survey and new perspectives," *Foundations and Trends in Information Retrieval*, vol. 14, no. 1, pp. 1–101, 2020.
- [15] T. Miller, "Explanation in artificial intelligence: Insights from the social sciences," *Artificial Intelligence*, vol. 267, pp. 1–38, 2019.
- [16] Anthropic, "Claude Opus 4.7 model card," Anthropic, 2026. [Online]. Available: <https://www.anthropic.com>
- [17] Microsoft Playwright, "Playwright for Python documentation," Microsoft Corp., 2024. [Online]. Available: <https://playwright.dev/python/>
- [18] R. T. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," *IETF RFC 7231*, 2014.

9. Appendix — Code Listings

9.1 ETL Pipeline Orchestrator (run_pipeline.py — excerpt)

```
def run_pipeline(categories: list[str]) -> dict:
    """Drive scrapers, then normalize. Always continues on per-source
    failure."""
    raw = {}
    for cat in categories:
        for src in SOURCE_BY_CATEGORY[cat]:
            try:
                raw.setdefault(cat, []).extend(SCRAPERS[src](cat))
            except Exception as e:
                logger.warning("scraper %s/%s failed: %s", src, cat, e)
                continue
    return normalize.run(raw)
```

Listing 1: Top-level pipeline orchestration. Scrapes each category from each source, tolerates per-source failures, then defers to the normalizer for canonicalization.

9.2 Delivery Normalization (normalize.py — excerpt)

```
def parse_delivery(text: str | None) -> int | None:
    """'Get it Tue, May 21' → days from today, else None."""
    if not text:
        return None
    m = DATE_RE.search(text)
    if not m:
        return None
    target = parse_date(m.group(1))
    return max(0, (target - date.today()).days)
```

Listing 2: Delivery-string parser. Returns None on unparseable text; the ranker treats None as a neutral 0.5 sub-score so missing data does not bias ranking.

9.3 Constraint-Aware Ranking (recommendation_engine_refactored.py — excerpt)

```
def recommend_products(prefs: dict, top_n: int = 10) -> list[dict]:
    """Filter → rank → top_n; relax constraints if too few survive."""
    products = load_products()

    result = _try_ranked(products, prefs, top_n)
    if result is not None:
        # full constraints
        return result

    # Fallback 1: drop delivery
    relaxed = {k: v for k, v in prefs.items() if k != "delivery_days_max"}
    result = _try_ranked(products, relaxed, top_n)
    if result is not None:
        return result

    # Fallback 2: drop price + delivery (category only)
    relaxed = {k: v for k, v in prefs.items()
               if k not in ("delivery_days_max", "price_max")}
```

```

result = _try_ranked(products, relaxed, top_n)
if result is not None:
    return result

# Fallback 3: ignore category – top by shared score
base = {"priority": prefs.get("priority", "balanced")}
return rank_products(load_products(), base)[:top_n]

```

Listing 3: Three-tier fallback. Each tier relaxes one constraint; the ranker never returns an empty list as long as the dataset is non-empty.

9.4 Scoring Rule (water_bottle gym example)

```

def _score_water_bottle(product, prefs, features):
    score, matches = 0.0, []
    if prefs.get("use_case") == "gym":
        score = _apply_rule(score, matches,
            features["lightweight"] or _title_has(product, "freesip", "owala"),
            "gym_suitable")
        score = _apply_rule(score, matches, features["insulated"],
            "insulated_gym")
    if prefs.get("insulated"):
        score = _apply_rule(score, matches, features["insulated"],
            "insulated", penalty=_SOFT_PENALTY)
    return score, matches

```

Listing 4: Category-specific scoring for the water-bottle gym use case. Bonuses are +0.10 per matched rule; soft penalties are -0.05 per missed rule the user requested.

9.5 FastAPI Routes (main.py — excerpt)

```

@app.post("/session/start")
def start_session(body: StartBody):
    try:
        parsed = claude_client.parse_initial_input(body.text)
        category = parsed.get("category", "unknown")
        preferences = parsed.get("preferences", {})
    except Exception:
        category, preferences = "unknown", {}    # graceful degradation

    if category in ("unknown", ""):
        return {"session_id": s["session_id"], "category": None,
            "next_question": None, "chips": CATEGORY_CHIPS}
    return {"session_id": s["session_id"], "category": category,
        "next_question": _next_question(s), "chips": None}

```

Listing 5: Session-start endpoint. The try/except keeps the demo functional when no Anthropic API key is available — chip selection becomes the fallback path.

9.6 Trust Boundary in the Recommendation Pipeline

```

def _run_recommendations(session: dict) -> list[dict]:
    prefs = {**session["preferences"], "category": session["category"]}
    products = engine.recommend_products(prefs, top_n=12)    # deterministic
    try:

```

```

        explanations = claude_client.write_explanations(
            products, session["preferences"], session["raw_input"])
    except Exception:
        explanations = [p.get("_explanation", "") for p in products] #
    fallback
    return [_format_product(p, explanations[i]) for i, p in
            enumerate(products)]

```

Listing 6: Where the trust boundary lives in code. Ranking is a pure function call; explanation generation is best-effort and gracefully falls back to the deterministic explainer.

9.7 Curated 3-Pick Differentiation (curated_picks excerpt)

```

def curated_picks(ranked, category):
    """Return Best Fit, Budget Pick, Stretch – with guard rails."""
    best = ranked[0]
    picks = [(best, "Best Fit",
               "Highest match score across all your stated preferences.")]

    # Budget Pick: cheapest in top-quartile that still scores >= 75% of best.
    score_floor = (best.get("_score") or 0.0) * 0.75
    budget_pool = sorted(
        [p for p in ranked[1:] if (p.get("_score") or 0.0) >= score_floor],
        key=lambda p: p.get("price") or 1e9)
    if budget_pool and budget_pool[0]["price"] + 5 <= best["price"]:
        saving = best["price"] - budget_pool[0]["price"]
        picks.append((budget_pool[0], "Budget Pick",
                      f"${budget_pool[0]['price']:.0f} – saves ${saving:.0f} vs
Best Fit.))

    # Stretch Pick: category-specific, requires a meaningful margin over Best
    Fit.
    if category == "smart_display":
        with_screen = [p for p in ranked if p.get("screen_inches")]
        if with_screen:
            cand = max(with_screen, key=lambda p: p["screen_inches"])
            if cand["screen_inches"] >= (best.get("screen_inches") or 0) + 2:
                picks.append((cand, "Large Screen Pick",
                              f'{cand["screen_inches"]}' – easier from across
the kitchen.))
    # ... water_bottle / kitchen_organizer branches similar ...
    return [{**p, "pick_label": l, "pick_reason": r} for p, l, r in picks]

```

Listing 7: Differentiated three-pick selection. The score_floor and margin tests prevent a \$5 sensor from posing as a Budget Pick and prevent a minor-difference product from claiming the Stretch slot.

9.8 Preference Match Checklist (per-product evaluator)

```

def _preference_checks(p, prefs):
    """Return [{label, status: match|miss|unknown}] for every user-picked
    chip."""
    rows = []
    cat = p.get("category")
    feats = p.get("_inferred_features") or {}
    rules = set(p.get("_category_rule_matches") or [])

```

```

# Use case fit: did the cat-specific rule for this use case actually fire?
use_case = prefs.get("use_case")
if use_case:
    wb_rule = {"gym": "gym_suitable", "outdoor": "outdoor_capacity",
               "kids": "kids_design", "daily": "daily_use"}
    matched = any([
        cat == "water_bottle" and wb_rule.get(u) in rules,
        cat == "smart_display" and u == "cooking"          and
        "kitchen_hub_or_recipe" in rules,
        cat == "smart_display" and u == "family"          and
        "family_scheduling"    in rules,
        cat == "smart_display" and u == "entertainment" and
        "entertainment_features" in rules,
    ] for u in (use_case if isinstance(use_case, list) else [use_case]))
    rows.append(_check("Use case fit", "match" if matched else "unknown",
                      ", ".join(use_case if isinstance(use_case, list)
                                else [use_case])))

# ... price_max, material, drinking_style, insulated, size, ecosystem,
#      screen_size, no_camera, organizer_material, visibility – each a
row ...
return rows

```

Listing 8: Per-pick preference checklist. Status comes from the same rule_matches the ranker used, so a row can only say 'match' if the rule fired.

9.9 Deterministic Refine Parser (regex fallback)

```

_SUPP_RULES_GLOBAL = [
    (r"(?:under|less than|below|cheaper than)\s*\$?\s*(\d+)", "price_max",
     "_int"),
    (r"\bno\s+camera\b|\bwithout\s+camera\b", "privacy_camera",
     "no_camera"),
    (r"\b(?:works\s+with\s+)?google\b|\bnest\s+hub\b", "voice_ecosystem",
     "google"),
    (r"\bleak[-\s]?proof\b|\bno\s+(?:spill|leak)\b",
     "leak_proof_preferred", True),
    # ... 25 more patterns ...
]

def _deterministic_parse_supplement(text, category):
    """Pattern-match a refine string into the same shape Claude would emit."""
    lower = (text or "").lower()
    updates = {}
    for pat, key, val in _SUPP_RULES_GLOBAL + _SUPP_RULES_BY_CAT.get(category,
    []):
        if key in updates:
            continue
        m = re.search(pat, lower)
        if not m:
            continue
        updates[key] = int(m.group(1)) if val == "_int" else val
    return {"preference_updates": updates,

```

```
"ai_response": "Got it – updating " + ", ".join(updates) + "."}
```

Listing 9: Regex fallback parser. Three categories of phrase are handled — global (price, delivery, privacy, ecosystem), and per-category (size, material, drinking style, structure, visibility). Each Claude call has a parallel fallback like this; the demo runs end-to-end with ANTHROPIC_API_KEY=placeholder.